

Laboratory 05

Deadlock Detection & Watchdog Timers

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXM236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	3 hours
CLO mapping	CLO 4

1. Objectives

By completing this lab you will be able to:

1. **Induce** a deadlock between two tasks holding two mutexes in opposite order.
2. **Prevent** deadlock using the resource-ordering convention.
3. **Configure** the MCXM236 hardware watchdog (WDOG32) and observe a reset.
4. **Implement** a software watchdog pattern using task notifications.
5. **Isolate** a hung task — log its identity and trigger a controlled reset.

2. Prerequisites

- ☐ Lab 04 complete — mutexes and SystemView working.
- ☐ ARM GNU Toolchain and pyocd working.

3. Background

3.1. Deadlock and the Four Coffman Conditions

A deadlock requires all four conditions simultaneously:

1. **Mutual exclusion** — resources are non-shareable.
2. **Hold and wait** — a task holds at least one resource while waiting for another.
3. **No preemption** — resources are released only voluntarily.
4. **Circular wait** — a cycle exists in the resource-allocation graph.

FreeRTOS mutexes satisfy conditions 1–3 by design. Your task is to prevent condition 4 using **resource ordering**: assign a global lock order and always acquire resources in ascending order.

3.2. Hardware Watchdog

The MCXM236 WDOG32 generates a system reset if not refreshed within a configured window. This is the last line of defence against a hung system.

4. Project Structure

```
lab05_watchdog/
  Makefile
  linker_MCXM236.ld
  src/
    main.c                <- task creation, WDOG init
    deadlock_tasks.c/.h   <- Parts A & B
    watchdog.c/.h         <- Parts C & D hardware + software watchdog
    FreeRTOSConfig.h
    uart.h / uart.c
    FreeRTOS-Kernel/
```

5. Part A — Induce a Deadlock

5.1. Step 1 — Implement the Deadlock Scenario

Create two mutexes and two tasks that acquire them in opposite order:

```

1  /* main.c */
2  SemaphoreHandle_t xMutexA;
3  SemaphoreHandle_t xMutexB;
4
5  xMutexA = xSemaphoreCreateMutex();
6  xMutexB = xSemaphoreCreateMutex();
7
8
9
10
11
12
13
14
15
16
17  /* deadlock_tasks.c */
18  void vTaskAlpha(void *pv) /* prio 2 */
19  {
20      for (;;) {
21          vTaskDelay(pdMS_TO_TICKS(10));
22          uart_printf("[A] taking MutexA...\r\n");
23          xSemaphoreTake(xMutexA, portMAX_DELAY);
24          uart_printf("[A] took MutexA. Taking MutexB...\r\n");
25          vTaskDelay(pdMS_TO_TICKS(5));
26          xSemaphoreTake(xMutexB, portMAX_DELAY);
27          uart_printf("[A] took both -- doing work\r\n");
28          xSemaphoreGive(xMutexB);
29          xSemaphoreGive(xMutexA);
30      }
31  }
32
33  void vTaskBeta(void *pv) /* prio 2 */
34  {
35      for (;;) {
36          vTaskDelay(pdMS_TO_TICKS(10));
37          uart_printf("[B] taking MutexB...\r\n");
38          xSemaphoreTake(xMutexB, portMAX_DELAY);
39          uart_printf("[B] took MutexB. Taking MutexA...\r\n");
40          vTaskDelay(pdMS_TO_TICKS(5));
41          xSemaphoreTake(xMutexA, portMAX_DELAY); /* <- opposite
42              order to Task A */
43          uart_printf("[B] took both -- doing work\r\n");
44          xSemaphoreGive(xMutexA);
45          xSemaphoreGive(xMutexB);
46      }
47  }

```

5.2. Step 2 — Build, Flash, and Observe

```
mingw32-make setup && mingw32-make && mingw32-make flash
```

Watch the terminal. Within a few seconds, both [A] and [B] will stop printing “took both” — the system is deadlocked.

✓ Checkpoint A

UART capture showing the last lines before deadlock, with both tasks blocked.

? Question A1

Draw the resource allocation graph at the moment of deadlock. Label each edge as an assignment or a request edge. Which Coffman condition does the circular edge represent?

? Question A2

The deadlock is **non-deterministic** — it does not always happen on the first iteration. What determines how quickly it manifests? Would the deadlock still be possible if both tasks ran at different priorities?

6. Part B — Prevent with Resource Ordering

6.1. Step 1 — Apply Resource Ordering

Assign a global lock order constant and enforce it in both tasks:

```

1  /* deadlock_tasks.c -- FIXED */
2  #define LOCK_ORDER_MUTEX_A  1
3  #define LOCK_ORDER_MUTEX_B  2
4  /* Rule: always acquire in ascending order */
5
6  /* vTaskAlpha unchanged -- already acquires A then B (correct order)
   * /
7
8  void vTaskBeta(void *pv)    /* FIXED: acquire A before B */
9  {
10     for (;;) {
11         vTaskDelay(pdMS_TO_TICKS(10));
12         uart_printf("[B] taking MutexA (order fix)...\r\n");
13         xSemaphoreTake(xMutexA, portMAX_DELAY);    /* A first */
14         uart_printf("[B] taking MutexB...\r\n");
15         xSemaphoreTake(xMutexB, portMAX_DELAY);    /* B second */
16         uart_printf("[B] took both -- doing work\r\n");
17         xSemaphoreGive(xMutexB);
18         xSemaphoreGive(xMutexA);
19     }
20 }
```

6.2. Step 2 — Verify No Deadlock

Run for at least 60 seconds. Both tasks should continue printing “took both” indefinitely.

✓ Checkpoint B

UART capture of 10+ seconds of normal operation with no deadlock.

? Question B1

Resource ordering prevents deadlock by breaking which Coffman condition? Explain precisely why a total order on acquisition prevents a cycle in the resource-allocation graph.

? Question B2

Your system has 6 mutexes protecting 6 different subsystems. How do you choose the lock order? What property must the ordering have? Is it always possible?

7. Part C — Hardware Watchdog

7.1. Step 1 — Configure WDOG32

The MCMXN236 WDOG32 resets the chip if not refreshed within a timeout window.

```

1  /* watchdog.c */
2  #include "fsl_wdog32.h"
3
4  void WDOG_Init_1s(void)
5  {
6      wdog32_config_t cfg;
7      WDOG32_GetDefaultConfig(&cfg);
8      cfg.timeoutValue = 0x100U;    /* ~1 second at LPO 128 Hz */
9      cfg.enableWdog32 = true;
10     cfg.clockSource = kWDOG32_ClockSourceLpoClk;
11     cfg.prescaler    = kWDOG32_ClockPrescalerDivide1;
12     WDOG32_Init(WDOG0, &cfg);
13 }
14
15 void WDOG_Kick(void)
16 {
17     WDOG32_Refresh(WDOG0);
18 }

```

7.2. Step 2 — Normal Operation

Create a high-priority watchdog task that kicks every 500 ms:

```

1  void vWatchdogTask(void *pv) /* prio 4 -- highest */
2  {
3      WDOG_Init_1s();
4      for (;;) {
5          vTaskDelay(pdMS_TO_TICKS(500));
6          WDOG_Kick();
7          uart_printf("[WDG] kicked\r\n");
8      }
9  }

```

7.3. Step 3 — Trigger the Watchdog Reset

Add a `#define SIMULATE_HANG 1` build flag. When set, `vWatchdogTask` sleeps for 2 seconds instead of kicking:

```

1  #ifdef SIMULATE_HANG
2      vTaskDelay(pdMS_TO_TICKS(2000)); /* miss deadline -> watchdog
                                         fires */
3  #else
4      WDOG_Kick();
5  #endif

```

Build with `make HANG=1` && `make HANG=1 flash`.

In `main.c`, before the scheduler starts, check the reset cause:

```

1  if (WDOG32_GetStatusFlags(WDOG0) & kWDOG32_InterruptFlag) {
2      uart_printf("[BOOT] Reset caused by watchdog!\r\n");
3  }

```

✓ Checkpoint C

UART capture showing [BOOT] Reset caused by watchdog! on the reboot following the simulated hang.

? Question C1

The WDOG32 uses the LPO (Low-Power Oscillator) at 128 Hz. Calculate the exact timeout in milliseconds for `timeoutValue = 0x100`. Why is a low-power oscillator used rather than the main system clock?

? Question C2

A **window watchdog** adds a minimum refresh time — you cannot kick too early OR too late. How would you configure a window watchdog to catch a “too-fast” kick? Sketch the `cfg.windowValue` setting.

8. Part D — Software Watchdog with Task Monitoring

8.1. Step 1 — Multi-Task Monitoring Pattern

The hardware watchdog is kicked by a dedicated monitor task that only kicks if **all** worker tasks have checked in during the last interval:

```

1  /* watchdog.c -- software monitor */
2  #define NUM_WORKERS 3
3  static volatile uint8_t worker_alive[NUM_WORKERS];
4
5  void wdog_checkin(uint8_t worker_id)
6  {
7      worker_alive[worker_id] = 1;
8  }
9
10 void vWatchdogMonitor(void *pv) /* prio 4 */
11 {
12     WDOG_Init_1s();
13     for (;;) {
14         vTaskDelay(pdMS_TO_TICKS(500));
15         bool all_ok = true;
16         for (int i = 0; i < NUM_WORKERS; i++) {
17             if (!worker_alive[i]) {
18                 uart_printf("[WDG] worker %d silent -- NOT kicking!\r\n", i);
19                 all_ok = false;
20             }
21             worker_alive[i] = 0;
22         }
23         if (all_ok) {
24             WDOG_Kick();
25             uart_printf("[WDG] all workers OK -- kicked\r\n");
26         }
27     }
28 }
29
30 void vWorkerTask(void *pv)
31 {
32     uint8_t id = (uint8_t)(uintptr_t)pv;
33     for (;;) {
34         vTaskDelay(pdMS_TO_TICKS(200));

```

```

35     wdog_checkin(id);
36     uart_printf("[W%d] working\r\n", id);
37 }
38 }

```

8.2. Step 2 — Simulate a Hanging Worker

Add a flag to stop one worker checking in after 5 seconds:

```

1  if (id == 1 && xTaskGetTickCount() > pdMS_TO_TICKS(5000)) {
2      for (;;) {} /* hang -- no checkin */
3  }

```

After the hang, the monitor stops kicking within 500 ms, and the watchdog fires ≈ 1 second later.

✓ Checkpoint D

UART capture showing the monitor detecting the silent worker, then the board reset and reboot message.

? Question D1

In this pattern, the monitor task itself could hang. How would you guard against that? (Hint: think about who monitors the monitor.)

? Question D2

The worker uses a volatile `uint8_t` flag rather than a task notification. What are the pros and cons of each approach? Which is safer in an RTOS context and why?

9. Deliverables

#	Item	Details
1	Part A UART capture	Deadlock state visible
2	Part A RAG diagram	Resource-allocation graph (hand-drawn or tool)
3	Part B UART capture	60+ seconds of normal operation
4	Part C UART capture	Watchdog reset cause message on reboot
5	Part D UART capture	Monitor detecting silent worker + reset
6	Written answers	Questions A1–A2, B1–B2, C1–C2, D1–D2

10. Key Concepts Demonstrated

Concept	Where you saw it
Deadlock induction — opposite lock order	Part A
Coffman condition 4 — circular wait	Part A diagram
Resource ordering — deadlock prevention	Part B
WDOG32 hardware watchdog configuration	Part C
Reset cause detection at boot	Part C
Software watchdog with per-task monitoring	Part D

11. Reference

Resource	Relevant sections
Coffman, Elphick & Shoshani (1971)	“System deadlocks” (<i>ACM Computing Surveys</i>)
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 6 (mutexes)
NXP MCXN236 Reference Manual	Chapter WDOG32
NXP MCUXpresso SDK	<code>fsl_wdog32.h</code> , <code>WDOG32_GetDefaultConfig</code>
